

```
1  import csv
2  import hashlib
3  import time
4  import statistics
5
6  STUDENT_FILE = "students.csv"
7  COURSE_FILE = "courses.csv"
8  PROFESSOR_FILE = "professors.csv"
9  LOGIN_FILE = "login.csv"
10
11 def encrypt_password(password):
12     return hashlib.sha256(password.encode()).hexdigest()
13
14
15 def read_csv(file):
16     data = []
17     try:
18         with open(file, 'r', newline='') as f:
19             reader = csv.DictReader(f)
20             for row in reader:
21                 data.append(row)
22     except FileNotFoundError:
23         pass
24     return data
25
26
27 def write_csv(file, fieldnames, data):
28     with open(file, 'w', newline='') as f:
29         writer = csv.DictWriter(f, fieldnames=fieldnames)
```

```

30         writer.writeheader()
31         writer.writerows(data)
32
33     class Student:
34
35         def display_records(self):
36             students = read_csv(STUDENT_FILE)
37             for s in students:
38                 print(s)
39
40         def add_new_student(self):
41             email = input("Email: ")
42             first = input("First Name: ")
43             last = input("Last Name: ")
44             course = input("Course ID: ")
45             grade = input("Grade: ")
46             marks = input("Marks: ")
47
48             students = read_csv(STUDENT_FILE)
49
50             students.append({
51                 "Email_address": email,
52                 "First_name": first,
53                 "Last_name": last,
54                 "Course_id": course,
55                 "Grade": grade,
56                 "Marks": marks
57             })

```

```

59     write_csv(STUDENT_FILE,
60               ["Email_address", "First_name", "Last_name", "Course_id", "Grade", "Marks"],
61               students)
62
63     def delete_student(self):
64         email = input("Enter email to delete: ")
65         students = read_csv(STUDENT_FILE)
66         students = [s for s in students if s["Email_address"] != email]
67
68         write_csv(STUDENT_FILE,
69                   ["Email_address", "First_name", "Last_name", "Course_id", "Grade", "Marks"],
70                   students)
71
72     def update_student_record(self):
73         email = input("Enter email to update: ")
74         students = read_csv(STUDENT_FILE)
75
76         for s in students:
77             if s["Email_address"] == email:
78                 s["Marks"] = input("New marks: ")
79                 s["Grade"] = input("New grade: ")
80
81         write_csv(STUDENT_FILE,
82                   ["Email_address", "First_name", "Last_name", "Course_id", "Grade", "Marks"],
83                   students)
84
85     def search_student(self):

```

```

86     email = input("Enter email to search: ")
87     students = read_csv(STUDENT_FILE)
88
89     start = time.time()
90
91     for s in students:
92         if s["Email_address"] == email:
93             print(s)
94
95     end = time.time()
96     print("Search time:", end - start)
97
98     def statistics(self):
99         students = read_csv(STUDENT_FILE)
100         marks = [int(s["Marks"]) for s in students if s["Marks"].isdigit()]
101
102         if marks:
103             print("Average:", sum(marks)/len(marks))
104             print("Median:", statistics.median(marks))
105
106     class Course:
107
108         def display_courses(self):
109             courses = read_csv(COURSE_FILE)
110             for c in courses:
111                 print(c)
112

```

```
113     def add_new_course(self):
114         cid = input("Course ID: ")
115         name = input("Course Name: ")
116         desc = input("Description: ")
117
118         courses = read_csv(COURSE_FILE)
119
120         courses.append({
121             "Course_id": cid,
122             "Course_name": name,
123             "Description": desc
124         })
125
126         write_csv(COURSE_FILE,
127                  ["Course_id", "Course_name", "Description"],
128                  courses)
129
130     class Professor:
131
132         def display_professors(self):
133             profs = read_csv(PROFESSOR_FILE)
134             for p in profs:
135                 print(p)
136
137         def add_new_professor(self):
138             email = input("Professor Email: ")
139             name = input("Name: ")
140             rank = input("Rank: ")
```

```

140     rank = input("Rank: ")
141     course = input("Course ID: ")
142
143     profs = read_csv(PROFESSOR_FILE)
144
145     profs.append({
146         "Professor_id": email,
147         "Professor_Name": name,
148         "Rank": rank,
149         "Course_id": course
150     })
151
152     write_csv(PROFESSOR_FILE,
153              ["Professor_id", "Professor_Name", "Rank", "Course_id"],
154              profs)
155
156 class LoginUser:
157
158     def register(self):
159         email = input("Email: ")
160         password = input("Password: ")
161         role = input("Role (student/professor): ")
162
163         encrypted = encrypt_password(password)
164
165         users = read_csv(LOGIN_FILE)
166
167         users.append({
168             "User_id": email,
169             "Password": encrypted,
170             "Role": role
171         })
172
173         write_csv(LOGIN_FILE, ["User_id", "Password", "Role"], users)

```

```

174
175     def login(self):
176         email = input("Email: ")
177         password = input("Password: ")
178
179         encrypted = encrypt_password(password)
180
181         users = read_csv(LOGIN_FILE)
182
183         for u in users:
184             if u["User_id"] == email and u["Password"] == encrypted:
185                 print("Login successful")
186                 return True
187
188         print("Login failed")
189         return False
190
191     def main_menu():
192
193         student = Student()
194         course = Course()
195         professor = Professor()
196
197         while True:
198             print("\nCheckMyGrade Menu")
199             print("1. Show Students")
200             print("2. Add Student")
201             print("3. Delete Student")
202             print("4. Update Student")
203             print("5. Search Student")
204             print("6. Student Statistics")
205             print("7. Show Courses")
206             print("8. Add Course")
207             print("9. Show Professors")
208             print("10. Add Professor")

```

```

208     print("10. Add Professor")
209     print("11. Exit")
210
211     input = input("Enter your number: ")
212
213     if input == "1":
214         student.display_records()
215
216     elif input == "2":
217         student.add_new_student()
218
219     elif input == "3":
220         student.delete_student()
221
222     elif input == "4":
223         student.update_student_record()
224
225     elif input == "5":
226         student.search_student()
227
228     elif input == "6":
229         student.statistics()
230
231     elif input == "7":
232         course.display_courses()
233
234     elif input == "8":
235         course.add_new_course()
236
237     elif input == "9":
238         professor.display_professors()
239
240     elif input == "10":
241         professor.add_new_professor()
242

```

```

243         elif input == "11":
244             break
245
246     else:
247         print("Invalid option")
248
249     login = LoginUser()
250
251     print("1. Register")
252     print("2. Login")
253
254     c = input("Enter your number: ")
255
256     if c == "1":
257         login.register()
258
259     if login.login():
260         main_menu()
261

```

TESTING

```

1 import unittest
2 import csv
3 import time
4 import os
5
6 class TestCheckMyGrade(unittest.TestCase):
7
8     def setUp(self):
9         # Reset students file
10        with open("students.csv", "w", newline='') as f:
11            writer = csv.writer(f)
12            writer.writerow(["Email_address", "First_name", "Last_name", "Course_id", "Grade", "Marks"])
13
14        # Reset courses file
15        with open("courses.csv", "w", newline='') as f:
16            writer = csv.writer(f)
17            writer.writerow(["Course_id", "Course_name", "Description"])
18
19        # Reset professors file
20        with open("professors.csv", "w", newline='') as f:
21            writer = csv.writer(f)
22            writer.writerow(["Professor_id", "Professor_Name", "Rank", "Course_id"])
23
24    def generate_students(self, n=1000):
25        with open("students.csv", "a", newline='') as f:
26            writer = csv.writer(f)
27
28            for i in range(n):
29                writer.writerow([
30                    f"user{i}@test.com",
31                    "Test",
32                    "User",
33                    "DATA200",
34                    "A",
35                    str(50 + (i % 50))
36                ])

```



```
37
38     def test_add_students(self):
39         self.generate_students(1000)
40
41         with open("students.csv", "r") as f:
42             data = list(csv.reader(f))
43
44         print("Total students:", len(data) - 1)
45         self.assertTrue(len(data) - 1 == 1000)
46
47     def test_search_time(self):
48         self.generate_students(1000)
49
50         start = time.time()
51         found = False
52
53         with open("students.csv", "r") as f:
54             reader = csv.DictReader(f)
55             for row in reader:
56                 if row["Email_address"] == "user500@test.com":
57                     found = True
58                     break
59
60         end = time.time()
61
62         print("Search time:", end - start)
63         self.assertTrue(found)
64
65     def test_sort_students_ascending(self):
66         self.generate_students(1000)
67
68         start = time.time()
69
70         with open("students.csv", "r") as f:
71             data = list(csv.DictReader(f))
```

```

72
73     sorted_data = sorted(data, key=lambda x: int(x["Marks"]))
74
75     end = time.time()
76
77     print("Sorting (ascending) time:", end - start)
78     self.assertTrue(len(sorted_data) == 1000)
79
80     def test_sort_students_descending(self):
81         self.generate_students(1000)
82
83         start = time.time()
84
85         with open("students.csv", "r") as f:
86             data = list(csv.DictReader(f))
87
88         sorted_data = sorted(data, key=lambda x: int(x["Marks"]), reverse=True)
89
90         end = time.time()
91
92         print("Sorting (descending) time:", end - start)
93         self.assertTrue(len(sorted_data) == 1000)
94
95     def test_delete_student(self):
96         self.generate_students(1000)
97
98         email_to_delete = "user10@test.com"
99
100        with open("students.csv", "r") as f:
101            data = list(csv.DictReader(f))
102
103        new_data = [row for row in data if row["Email_address"] != email_to_delete]
104
105        with open("students.csv", "w", newline='') as f:

```

```

106         writer = csv.DictWriter(f, fieldnames=data[0].keys())
107         writer.writeheader()
108         writer.writerows(new_data)
109
110     self.assertTrue(len(new_data) == 999)
111
112     def test_modify_student(self):
113         self.generate_students(1000)
114
115         with open("students.csv", "r") as f:
116             data = list(csv.DictReader(f))
117
118         for row in data:
119             if row["Email_address"] == "user20@test.com":
120                 row["Marks"] = "99"
121
122         with open("students.csv", "w", newline='') as f:
123             writer = csv.DictWriter(f, fieldnames=data[0].keys())
124             writer.writeheader()
125             writer.writerows(data)
126
127         self.assertTrue(True)
128
129     def test_add_course(self):
130         with open("courses.csv", "a", newline='') as f:
131             writer = csv.writer(f)
132             writer.writerow(["TEST101", "Test Course", "Sample Description"])
133
134         with open("courses.csv", "r") as f:
135             data = list(csv.reader(f))
136
137         self.assertTrue(len(data) > 1)
138
139     def test_delete_course(self):

```

```

139     def test_delete_course(self):
140         with open("courses.csv", "a", newline='') as f:
141             writer = csv.writer(f)
142             writer.writerow(["TEST101", "Test Course", "Sample Description"])
143
144         with open("courses.csv", "r") as f:
145             data = list(csv.DictReader(f))
146
147         data = [row for row in data if row["Course_id"] != "TEST101"]
148
149         with open("courses.csv", "w", newline='') as f:
150             writer = csv.DictWriter(f, fieldnames=["Course_id", "Course_name", "Description"])
151             writer.writeheader()
152             writer.writerows(data)
153
154         self.assertTrue(True)
155
156     def test_add_professor(self):
157         with open("professors.csv", "a", newline='') as f:
158             writer = csv.writer(f)
159             writer.writerow(["prof@test.com", "Test Prof", "Senior", "DATA200"])
160
161         with open("professors.csv", "r") as f:
162             data = list(csv.reader(f))
163
164         self.assertTrue(len(data) > 1)
165
166     def test_delete_professor(self):
167         with open("professors.csv", "a", newline='') as f:
168             writer = csv.writer(f)
169             writer.writerow(["prof@test.com", "Test Prof", "Senior", "DATA200"])
170
171         with open("professors.csv", "r") as f:
172             data = list(csv.DictReader(f))
173
174         data = [row for row in data if row["Professor_id"] != "prof@test.com"]
175
176         with open("professors.csv", "w", newline='') as f:
177             writer = csv.DictWriter(f, fieldnames=["Professor_id", "Professor_Name", "Rank", "Course_id"])
178             writer.writeheader()
179             writer.writerows(data)
180
181         self.assertTrue(True)
182
183
184 if __name__ == "__main__":
185     unittest.main()

```

```
[Running] python -u "c:\Users\ARYAN MITTAL\Desktop\Data200\Lab1\checkmygrade.py"
..Total students: 1000
.....Search time: 0.0218203067779541
.Sorting (ascending) time: 0.01522064208984375
.Sorting (descending) time: 0.014468669891357422
.
-----
Ran 10 tests in 0.197s

OK

[Done] exited with code=0 in 1.583 seconds
```

UML DIAGRAM



